

Documentación del Componente: AdminDashboard

1. Descripción General

El componente **AdminDashboard** constituye el panel principal de administración del sistema A&L Compresores y Partes.

Su función es proporcionar una interfaz centralizada desde la cual los administradores pueden acceder a los diferentes módulos del sistema, gestionar productos, controlar inventario, visualizar reportes, administrar usuarios y monitorear notificaciones operativas.

2. Objetivo

Facilitar la gestión integral de la plataforma mediante una interfaz administrativa organizada, intuitiva y escalable.

3. Funcionalidades Principales

Gestión de Navegación

El panel permite navegar entre distintos módulos administrativos sin recargar la aplicación.

Las secciones disponibles son:

- Dashboard
 - Productos
 - Control de Stock
 - Proveedores
 - Bitácora
 - Usuarios
 - Reportes
 - Notificaciones
 - Mi Perfil
-

Gestión de Productos

Al iniciar el panel se realiza una consulta automática al servidor para obtener la información actualizada de los productos registrados.

La información obtenida es utilizada para:

- Visualización de inventario.
- Control de stock.
- Generación de notificaciones.
- Elaboración de reportes.

Sistema de Notificaciones

El sistema identifica automáticamente productos con bajo inventario.

Se considera una alerta cuando:

Stock < 10 unidades

Los productos suspendidos no son considerados dentro de las notificaciones.

Control de Sesión

El administrador puede cerrar sesión mediante la opción "Salir del Panel", ejecutando el proceso de cierre definido por la aplicación.

4. Componentes Integrados

Componente	Función
DashboardHome	Resumen general del sistema
Productos	Administración de productos
ControlStock	Gestión de inventario
Usuarios	Administración de usuarios
Reportes	Generación de reportes
Notificaciones	Alertas y eventos
MiPerfil	Gestión del perfil administrativo
Bitacora	Registro de actividades

5. Variables de Estado

seccionActiva

Controla la sección actualmente visible dentro del panel.

Valores posibles:

- dashboard
 - productos
 - stock
 - proveedores
 - bitacora
 - usuarios
 - reportes
 - notificaciones
 - perfil
-

productos

Almacena el listado de productos recuperados desde el servidor.

Se utiliza para:

- Cálculo de alertas.
 - Reportes.
 - Inventario.
 - Estadísticas.
-

6. Flujo de Funcionamiento

Paso 1

El administrador accede al sistema.

Paso 2

Se carga automáticamente el panel principal.

Paso 3

El sistema consulta los productos registrados mediante la API.

Paso 4

Se calculan las notificaciones de inventario.

Paso 5

Se muestra el menú lateral de navegación.

Paso 6

El administrador selecciona una sección.

Paso 7

Se renderiza dinámicamente el módulo correspondiente.

7. Menú Administrativo

Dashboard

Presenta indicadores generales del sistema.

Productos

Permite crear, editar, consultar y eliminar productos.

Control de Stock

Permite monitorear existencias y movimientos de inventario.

Proveedores

Módulo reservado para la gestión de proveedores.

Actualmente se encuentra en desarrollo.

Bitácora

Permite consultar registros históricos de actividades realizadas dentro del sistema.

Usuarios

Gestiona cuentas y permisos de acceso.

Reportes

Genera información estadística y reportes operativos.

Notificaciones

Muestra alertas relacionadas con:

- Bajo inventario.
- Eventos importantes.

Mi Perfil

Permite gestionar la información del administrador.

8. Comunicación con el Backend

Endpoint utilizado

GET /productos

Objetivo

Obtener todos los productos registrados para alimentar los módulos administrativos.

Respuesta esperada

```
[  
  {  
    "id": 1,  
    "nombre": "Compresor Industrial",  
    "stock": 15  
  }  
]
```

9. Requerimientos Funcionales

RF-01

El sistema debe mostrar el panel administrativo.

RF-02

El sistema debe permitir cambiar entre módulos administrativos.

RF-03

El sistema debe consultar los productos desde el servidor.

RF-04

El sistema debe detectar productos con bajo inventario.

RF-05

El sistema debe mostrar notificaciones de stock.

RF-06

El sistema debe permitir cerrar sesión.

10. Requerimientos No Funcionales

RNF-01

La interfaz debe ser responsiva.

RNF-02

La navegación debe realizarse sin recargar la página.

RNF-03

La carga de información debe ejecutarse de forma asíncrona.

RNF-04

La interfaz debe mantener una identidad visual corporativa.

RNF-05

El sistema debe ser modular y escalable.

11. Arquitectura del Componente

AdminDashboard



- Notificaciones
 - MiPerfil
 - Proveedores (En desarrollo)
-

12. Resultado Esperado

El componente AdminDashboard centraliza todas las operaciones administrativas del sistema, permitiendo gestionar productos, usuarios, inventario y reportes desde una única interfaz, optimizando la administración y el control operativo de la plataforma A&L Compresores y Partes.

```

import { useState, useEffect } from "react";
import DashboardHome from "../AdminDashboard/DashboardHome";
import MiPerfil from "../AdminDashboard/MiPerfil";
import Productos from "../AdminDashboard/Productos";
import Notificaciones from "../AdminDashboard/Notificaciones";
import Reportes from "../AdminDashboard/Reportes";
import Usuarios from "../AdminDashboard/Usuarios";
import ControlStock from "../AdminDashboard/ControlStock";
import Bitacora from "../AdminDashboard/Bitacora";
const logoMarca =
"https://res.cloudinary.com/duvoqozcl/image/upload/v1777394217/logo-ayl.png";

function SeccionVacía({ nombre }) {
  return (
    <div className="p-5 text-center">
      <div className="display-1 opacity-25 mb-3">
        <i className="bi bi-gear"></i></div>
        <h2 className="fw-bold mb-1">{nombre}</h2>
        <p className="text-secondary">Esta sección está en fase de
desarrollo técnico.</p>
      </div>
    );
  }

const API_URL = "http://localhost:3001/productos";

function AdminDashboard({ logout }) {
  const [seccionActiva, setSeccionActiva] = useState("dashboard");

  const [productos, setProductos] = useState([]);

  const obtenerProductos = async () => {
    try {
      const res = await fetch(API_URL);
      if (!res.ok) throw new Error(`Error ${res.status}`);
      const data = await res.json();
      setProductos(Array.isArray(data) ? data : []);
    } catch (error) {

```



```

        console.error("Error al cargar productos del dashboard admin:",
error);
        setProductos([]);
    }
};

useEffect(() => {
    obtenerProductos();
}, []);

const notificaciones = productos.filter(p => !p.suspendido &&
Number(p.stock || 0) < 10).length;

const obtenerImagen = (img) => {
    if (!img) return "/src/assets/compresor.jpg";
    return img.startsWith("http") ? img : `/src/assets/${img}`;
};

const MENU_ITEMS = [
    { id: "dashboard", label: "Dashboard", icon: "bi-speedometer2",
badge: null },
    { id: "productos", label: "Productos", icon: "bi-archive", badge:
null },
    { id: "stock", label: "Control de Stock", icon:
"bi-graph-up-arrow", badge: 3 },
    { id: "proveedores", label: "Proveedores", icon: "bi-truck", badge:
null },
    { id: "bitacora", label: "Bitácora", icon: "bi-journal-text",
badge: null },
    { id: "usuarios", label: "Usuarios", icon: "bi-people", badge: null
},
    { id: "reportes", label: "Reportes", icon:
"bi-file-earmark-bar-graph", badge: null },
    { id: "notificaciones", label: "Notificaciones", icon: "bi-bell",
badge: notificaciones },
    { id: "perfil", label: "Mi Perfil", icon: "bi-gear", badge: null },
];

// --- RENDERIZADO DINÁMICO ---
function renderContenido() {

```

```

switch (seccionActiva) {
  case "dashboard":
    return <DashboardHome />;

  case "perfil":
    return <MiPerfil />;

  case "productos":
    return <Productos />;

  case "stock":
    return <ControlStock />;

  case "notificaciones":
    return (
      <div className="p-4">
        <Notificaciones
          productos={productos}
          obtenerImagen={obtenerImagen}
        />
      </div>
    );

  case "proveedores":
    return <SeccionVacía nombre="Proveedores" />;

  case "usuarios": return <Usuarios />;

  case "reportes":
    return <Reportes />;
  case "bitacora":
    return <Bitacora />;

  default: return <DashboardHome />;
}

return (
  <div className="d-flex min-vh-100 bg-light">

    {/* — SIDEBAR — */}

```

```

<aside
  className="border-end d-flex flex-column p-3 shadow-sm"
  style={{ width: 260, minWidth: 260, position: "sticky", top: 0,
height: "100vh", zIndex: 1000, backgroundColor: "#000000", borderColor:
"rgba(255,255,255,0.08)" }}
  >

    {/* Logo Corporativo */}
    <div className="d-flex align-items-center gap-2 pb-3 mb-3
border-bottom">
      {/* Contenedor del Logo */}
      <div
        className="d-flex align-items-center justify-content-center
rounded-3"
        style={{ width: "auto", height: 45, flexShrink: 0 }}
      >
        <img
          src={logoMarca}
          alt="A&L Compresores"
          style={{ height: "40px", width: "auto", objectFit:
"contain" }}
        />
      </div>

      <div>
        <div className="text-warning fw-bold" style={{ fontSize:
"0.7rem", letterSpacing: "1px" }}>
          SISTEMA ADMIN
        </div>
      </div>
    </div>

    {/* Info del Administrador */}
    <div className="d-flex align-items-center gap-2 bg-light
rounded-4 p-3 mb-4">
      <div
        className="d-flex align-items-center justify-content-center
bg-white shadow-sm rounded-circle text-dark"
        style={{ width: 42, height: 42, fontSize: "1.2rem",
flexShrink: 0 }}
      >
        <i className="bi bi-person-badge"></i> {/* <--- Cambia el
emoji por este icono */}

```

```

</div>
<div className="overflow-hidden">
  <div className="fw-bold text-truncate" style={{ fontSize:
"0.85rem" }}>Admin A&L</div>
  <span
    className="badge rounded-pill"
    style={{ background: "#121212", fontSize: "0.6rem",
fontWeight: 600 }}
  >
    Nivel Máster
  </span>
</div>
</div>

{/* Menú de Navegación */}
<nav className="d-flex flex-column gap-1 flex-grow-1">
  {MENU_ITEMS.map((item) => {
    const isActive = seccionActiva === item.id;
    return (
      <button
        key={item.id}
        onClick={() => setSeccionActiva(item.id)}
        className="d-flex align-items-center gap-3 border-0
rounded-3 px-3 py-2 w-100 text-start fw-semibold"
        style={{
          background: isActive ? "#F5A623" : "transparent",
          color: isActive ? "#000000" : "#E9ECEF",
          fontSize: "0.9rem",
          cursor: "pointer",
          transition: "all 0.25s cubic-bezier(0.4, 0, 0.2, 1)",
          boxShadow: isActive ? "0 4px 12px rgba(245, 166, 35,
0.3)" : "none"
        }}
      >
        {item.label}
      </button>
    )
  })}
  <button
    className="d-flex align-items-center gap-3 border-0
rounded-3 px-3 py-2 w-100 text-start fw-semibold"
    style={{
      background: "transparent",
      color: "#E9ECEF",
      fontSize: "0.9rem",
      cursor: "pointer",
      transition: "all 0.25s cubic-bezier(0.4, 0, 0.2, 1)",
      boxShadow: "0 4px 12px rgba(245, 166, 35, 0.3)"
    }}
    >
    Nivel Máster
  </button>
</nav>

```

```

        e.currentTarget.style.color = "#555";
        e.currentTarget.style.paddingLeft = "16px";
    }
    }}
    >
    <i className={`bi ${item.icon}`} style={{ fontSize:
"1.1rem" }}></i>
    <span className="flex-grow-1">{item.label}</span>

    {item.badge > 0 && (
    <span
    className="badge rounded-pill"
    style={{
        background: isActive ? "rgba(255,255,255,0.25)" :
"#dc3545",

        fontSize: "0.65rem",
        minWidth: 22
    }}
    >
        {item.badge}
    </span>
    )}
    </button>
    );
    }}}
</nav>

{/* Botón de Salida */}
<div className="border-top pt-3">
    <button
        onClick={() => logout()}
        className="btn btn-link d-flex align-items-center gap-2
w-100 text-decoration-none fw-bold"
        style={{ color: "#adb5bd", fontSize: "0.85rem", transition:
"0.3s" }}
        onMouseEnter={(e) => e.currentTarget.style.color =
"#F5A623"}
        onMouseLeave={(e) => e.currentTarget.style.color =
"#adb5bd"}
    >
        <span><i className="bi bi-door-open"></i></span> Salir del
Panel
    </button>

```

```

    </div>

</aside>

{/* — CONTENIDO — */}
<main className="flex-grow-1 overflow-auto bg-white">
  <div className="animate-fade-in">
    {renderContenido()}
  </div>
</main>

<style>{`
  @keyframes fadeIn {
    from { opacity: 0; transform: translateY(5px); }
    to { opacity: 1; transform: translateY(0); }
  }
  .animate-fade-in {
    animation: fadeIn 0.4s ease-out forwards;
  }
`}</style>

</div>
);
}

export default AdminDashboard;

```